Assignment 1 — Quản lý sinh viên bằng Dictionary

1. Đề bài tiếng Anh

ASSIGNMENT 1

Objective

Develop a menu-driven Python program to manage student information using a dictionary data structure and functions.

Requirements

Write a Python program that implements the following features:

1. **Data Structure:** Store students in a dictionary. Each student includes: `id`, `name`, `score`.
2. **Create (Add student):** Input student information. Check for duplicate ID before adding.
3. **Read (Display students):** Show all students in the system.
4. **Update student:** Update student information based on ID.
5. **Delete student:** Delete a student by ID. Require confirmation before deletion.
6. **Validate data:** Ensure valid input (non-empty name, valid score range, correct format).
7. **Check duplicate ID:** Do not allow adding students with existing IDs.
8. **Sorting:** Sort students by score (ascending/descending).
9. **Advanced statistics:** Highest score student, lowest score student, average score, count students by performance level.
10. **Academic classification:** Classify students based on score (e.g., Excellent, Good, Average, Poor).

Đề được chuyển từ `Ass 1.docx` sang Markdown; nội dung và thứ tự yêu cầu được giữ lại.

2. Đề bài tiếng Việt

BÀI TẬP 1 — Quản lý sinh viên bằng Dictionary (từ điển)

Mục tiêu

Xây dựng chương trình Python có menu quản lý thông tin sinh viên, sử dụng cấu trúc dữ liệu **dictionary** và các hàm.

Yêu cầu

Viết chương trình có đầy đủ các chức năng sau:

1. **Cấu trúc dữ liệu:** Lưu sinh viên bằng dictionary (từ điển), mỗi bản ghi có `id`, `name`, `score`.
2. **Thêm sinh viên (Create):** Nhập thông tin, kiểm tra mã ID trùng trước khi thêm.
3. **Hiển thị (Read):** Xuất toàn bộ sinh viên hiện có.
4. **Cập nhật (Update):** Sửa thông tin sinh viên dựa trên ID.
5. **Xóa (Delete):** Xóa theo ID và **phải có xác nhận** trước khi xóa.
6. **Kiểm tra dữ liệu:** Không cho phép tên rỗng, điểm ngoài khoảng hợp lệ hoặc sai định dạng.

7. **Chống trùng mã:** Không thêm được sinh viên có ID đã tồn tại.
8. **Sắp xếp:** Sắp xếp theo điểm tăng dần hoặc giảm dần.
9. **Thống kê nâng cao:** Tìm sinh viên có điểm cao nhất, thấp nhất, tính điểm trung bình và đếm sinh viên theo mức học lực.
10. **Phân loại học lực:** Dựa trên điểm, chia thành Xuất sắc (Excellent), Khá (Good), Trung bình (Average) và Yếu (Poor).

Lưu ý: Đề Word chỉ nêu ví dụ các mức học lực, **không quy định ngưỡng điểm cụ thể**. Các ngưỡng của code tham khảo phải được trình bày như quy ước triển khai, không phải dữ liệu gốc.

3. Code Python có chú thích

```
from collections import Counter  # Nạp thư viện (import: nạp vào) để sử dụng hàm có sẵn.
import sys  # Nạp thư viện (import: nạp vào) để sử dụng hàm có sẵn.


def validate(student_id, name, score):  # Khai báo hàm (function: hàm), nhóm các bước và cho phép gọi lại.
    student_id = student_id.strip()  # Bỏ khoảng trắng thừa (strip: cắt khoảng trắng).
    name = name.strip()  # Bỏ khoảng trắng thừa (strip: cắt khoảng trắng).
    if not student_id or not name:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        raise ValueError("Student ID and name are required.")  # Thông báo lỗi khi dữ liệu không hợp lệ.
    try:  # Thử thao tác có thể phát sinh lỗi.
        score = float(score)  # Tính hoặc cập nhật biến score từ biểu thức, giữ đúng công thức của bài.
    except (ValueError, TypeError) as error:  # Bắt ngoại lệ (exception: lỗi) để thông báo rõ.
        raise ValueError("Score must be numeric.") from error  # Thông báo lỗi khi dữ liệu không hợp lệ.
    if not 0 <= score <= 10:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        raise ValueError("Score must be between 0 and 10.")  # Thông báo lỗi khi dữ liệu không hợp lệ.
    return student_id, name, score  # Trả kết quả (return: trả về) cho hàm gọi.


def add(students, student_id, name, score):  # Khai báo hàm (function: hàm), nhóm các bước và cho phép gọi lại.
    student_id, name, score = validate(student_id, name, score)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    if student_id in students:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        raise ValueError(f"Duplicate student ID: {student_id}")  # Thông báo lỗi khi dữ liệu không hợp lệ.
    students[student_id] = {"name": name, "score": score}  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.


def display(students):  # Khai báo hàm (function: hàm), nhóm các bước và cho phép gọi lại.
    if not students:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        print("No students stored.")  # In kết quả (output: kết quả) để đối chiếu với đề.
    for student_id, record in students.items():  # Duyệt lần lượt dữ liệu bằng vòng lặp (loop: lặp).
        print(f"{student_id} | {record['name']} | "  # In kết quả (output: kết quả) để đối chiếu với đề.
              f"{record['score']:g}/10 | {classify(record['score'])}")  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.


def update(students, student_id, new_name, new_score):  # Khai báo hàm (function: hàm), nhóm các bước và cho phép gọi lại.
    if student_id not in students:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        raise KeyError(f"Unknown student ID: {student_id}")  # Thông báo lỗi khi dữ liệu không hợp lệ.
    _, name, score = validate(student_id, new_name, new_score)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    students[student_id] = {"name": name, "score": score}  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.


def delete(students, student_id, confirmed=False):  # Khai báo hàm (function: hàm), nhóm các bước và cho phép gọi lại.
    if student_id not in students:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        raise KeyError(f"Unknown student ID: {student_id}")  # Thông báo lỗi khi dữ liệu không hợp lệ.
```

```
    if not confirmed:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        return False  # Trả kết quả (return: trả về) cho hàm gọi.
    del students[student_id]  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    return True  # Trả kết quả (return: trả về) cho hàm gọi.


def classify(score):  # Khai báo hàm (function: hàm), nhóm các bước và cho phép gọi lại.
    if score >= 9:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        return "Excellent"  # Trả kết quả (return: trả về) cho hàm gọi.
    if score >= 7:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        return "Good"  # Trả kết quả (return: trả về) cho hàm gọi.
    if score >= 5:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        return "Average"  # Trả kết quả (return: trả về) cho hàm gọi.
    return "Poor"  # Trả kết quả (return: trả về) cho hàm gọi.


def sort_by_score(students, descending=False):  # Khai báo hàm (function: hàm), nhóm các bước và cho phép gọi lại.
    return sorted(  # Trả kết quả (return: trả về) cho hàm gọi.
        students.items(),  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
        key=lambda pair: pair[1]["score"],  # Tính hoặc cập nhật biến key từ biểu thức, giữ đúng công thức của bài.
        reverse=descending,  # Tính hoặc cập nhật biến reverse từ biểu thức, giữ đúng công thức của bài.
    )  # Đóng danh sách, biểu thức hoặc tập giá trị đang tạo.


def statistics(students):  # Khai báo hàm (function: hàm), nhóm các bước và cho phép gọi lại.
    if not students:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        return None  # Trả kết quả (return: trả về) cho hàm gọi.
    scores = [item["score"] for item in students.values()]  # Tính hoặc cập nhật biến scores từ biểu thức, giữ đúng công thức của bài.
    counts = Counter(classify(score) for score in scores)  # Tính hoặc cập nhật biến counts từ biểu thức, giữ đúng công thức của bài.
    return {  # Trả kết quả (return: trả về) cho hàm gọi.
        "highest": max(scores),  # Lưu trường tên khóa (key: khóa) và kết quả vào từ điển.
        "lowest": min(scores),  # Lưu trường tên khóa (key: khóa) và kết quả vào từ điển.
        "average": sum(scores) / len(scores),  # Lưu trường tên khóa (key: khóa) và kết quả vào từ điển.
        "classification_counts": dict(counts),  # Lưu trường tên khóa (key: khóa) và kết quả vào từ điển.
    }  # Đóng danh sách, biểu thức hoặc tập giá trị đang tạo.


def sample_students():  # Khai báo hàm (function: hàm), nhóm các bước và cho phép gọi lại.
    scores = [9.5, 8, 7.5, 6, 5.5, 4.5, 7, 9, 8.5, 6.5, 10, 5]  # Tính hoặc cập nhật biến scores từ biểu thức, giữ đúng công thức của bài.
    students = {}  # Tính hoặc cập nhật biến students từ biểu thức, giữ đúng công thức của bài.
    for number, score in enumerate(scores, start=1):  # Duyệt lần lượt dữ liệu bằng vòng lặp (loop: lặp).
        add(students, f"S{number:03d}", f"Learner {number:02d}", score)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    return students  # Trả kết quả (return: trả về) cho hàm gọi.


def run_demo():  # Khai báo hàm (function: hàm), nhóm các bước và cho phép gọi lại.
    students = sample_students()  # Tính hoặc cập nhật biến students từ biểu thức, giữ đúng công thức của bài.
    if not (len(students) == 12): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi khi dữ liệu không đạt.
    add(students, "S013", "New Learner", 8)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    update(students, "S013", "Updated Learner", 8.5)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    if not (delete(students, "S013", confirmed=False) is False): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi
khi dữ liệu không đạt.
    if not ("S013" in students): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi khi dữ liệu không đạt.
    if not (delete(students, "S013", confirmed=True) is True): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi khi
dữ liệu không đạt.
    if not ("S013" not in students): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi khi dữ liệu không đạt.
    if not (len(students) == 12): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi khi dữ liệu không đạt.
```

```
    print("Ascending scores (first 4):", sort_by_score(students)[:4])  # In kết quả (output: kết quả) để đối chiếu với đề.
    print("Descending scores (first 4):",  # In kết quả (output: kết quả) để đối chiếu với đề.
          sort_by_score(students, descending=True)[:4])  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    print("Statistics:", statistics(students))  # In kết quả (output: kết quả) để đối chiếu với đề.
    print("Student list:")  # In kết quả (output: kết quả) để đối chiếu với đề.
    display(students)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.


def interactive_menu():  # Khai báo hàm (function: hàm), nhóm các bước và cho phép gọi lại.
    students = sample_students()  # Tính hoặc cập nhật biến students từ biểu thức, giữ đúng công thức của bài.
    menu = "\n1.Display 2.Add 3.Update 4.Delete 5.Sort 6.Statistics 0.Exit"  # Tính hoặc cập nhật biến menu từ biểu thức, giữ đúng công thức của bài.
    while True:  # Lặp khi điều kiện còn đúng, dừng khi sai.
        print(menu)  # In kết quả (output: kết quả) để đối chiếu với đề.
        action = input("Choose: ").strip()  # Bỏ khoảng trắng thừa (strip: cắt khoảng trắng).
        try:  # Thử thao tác có thể phát sinh lỗi.
            if action == "0":  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
                break  # Thoát vòng lặp tại vị trí này.
            elif action == "1":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                display(students)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
            elif action == "2":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                add(students, input("ID: "), input("Name: "),  # Nhận input (dữ liệu nhập) rồi kiểm tra trước khi dùng.
                    input("Score [0,10]: "))  # Nhận input (dữ liệu nhập) rồi kiểm tra trước khi dùng.
            elif action == "3":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                key = input("ID to update: ").strip()  # Bỏ khoảng trắng thừa (strip: cắt khoảng trắng).
                update(students, key, input("New name: "), input("New score: "))  # Nhận input (dữ liệu nhập) rồi kiểm tra trước khi dùng.
            elif action == "4":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                key = input("ID to delete: ").strip()  # Bỏ khoảng trắng thừa (strip: cắt khoảng trắng).
                answer = input("Confirm deletion? [y/N]: ").strip().lower()  # Bỏ khoảng trắng thừa (strip: cắt khoảng trắng).
                if not delete(students, key, confirmed=(answer == "y")):  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
                    print("Deletion cancelled.")  # In kết quả (output: kết quả) để đối chiếu với đề.
            elif action == "5":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                descending = input("Descending? [y/N]: ").strip().lower() == "y"  # Bỏ khoảng trắng thừa (strip: cắt khoảng trắng).
                for key, item in sort_by_score(students, descending):  # Duyệt lần lượt dữ liệu bằng vòng lặp (loop: lặp).
                    print(key, item)  # In kết quả (output: kết quả) để đối chiếu với đề.
            elif action == "6":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                print(statistics(students))  # In kết quả (output: kết quả) để đối chiếu với đề.
            else:  # Xử lý trường hợp còn lại.
                print("Unknown menu option.")  # In kết quả (output: kết quả) để đối chiếu với đề.
        except (ValueError, KeyError) as error:  # Bắt ngoại lệ (exception: lỗi) để thông báo rõ.
            print("Input error:", error)  # In kết quả (output: kết quả) để đối chiếu với đề.


if __name__ == "__main__":  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
    if "--demo" in sys.argv:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        run_demo()  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    else:  # Xử lý trường hợp còn lại.
        interactive_menu()  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
```

4. Giải thích nội dung và cách giải

Câu 1

Dùng `dict` lồng nhau: khóa ngoài là ID duy nhất, giá trị là đối tượng có `name` và `score`. Hàm tạo dữ liệu giả lập để tránh thông tin cá nhân.

Vì sao dùng dictionary lồng nhau? ID là khóa duy nhất, giúp tìm một sinh viên trực tiếp mà không cần duyệt toàn bộ list; dữ liệu đi kèm có thể đặt trong dictionary con. **Các bước:** Khởi tạo từ điển → dùng ID làm khóa → lưu tên, điểm làm giá trị → truy cập khi cần.

```
students = {}
students["S01"] = {"name": "An", "score": 8.5}
name = students["S01"]["name"]
```

Cú pháp: `students["S01"]` truy cập bản ghi theo ID; `["name"]` truy cập trường tên. ID trùng sẽ **ghi đè** nếu gán trực tiếp, nên chức năng thêm ở Câu 2 phải kiểm tra trước. Tìm theo khóa trung bình O(1), không có nghĩa luôn tuyệt đối cố định.

Điểm cần kiểm tra: dictionary (từ điển) là cấu trúc lưu trữ chính; ID là khóa.

Ví dụ, cách kiểm tra và lỗi dễ gặp: ID là khóa duy nhất của từ điển; mỗi khóa dẫn tới thông tin người học. Dùng tên làm khóa dễ ghi đè khi trùng tên.

Thuật ngữ cần nhớ: Dictionary (từ điển).

Câu 2

Đọc các trường, chuẩn hóa bằng `.strip()`, đổi điểm bằng `float()`, kiểm tra miền 0–10, rồi kiểm tra ID chưa có trước khi thêm.

Vì sao xác thực trước khi thêm? Nếu ID trùng, dictionary có thể ghi đè sinh viên cũ; điểm ngoài 0–10 sẽ làm sai phân loại. **Các bước:** Làm sạch ID/tên → đổi điểm sang `float` → kiểm tra điểm → so ID đã tồn tại chưa → chỉ gán khi hợp lệ.

```
student_id = "S02".strip()
score = float("8.5")
if student_id in students:
    raise ValueError("ID đã tồn tại")
if not (0 <= score <= 10):
    raise ValueError("Điểm phải từ 0 đến 10")
students[student_id] = {"name": "Bình", "score": score}
```

Giải thích: `.strip` bỏ khoảng trắng; `float` chuyển chuỗi số; `in` kiểm tra khóa có tồn tại; `not` phủ định; `0 <= score <= 10` kiểm tra cả hai biên; `raise ValueError` báo dữ liệu sai, không âm thầm ghi đè.

Điểm cần kiểm tra: xác thực `id`, `name`, `score` và chặn trùng ID.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Kiểm tra ID đã có hay chưa trước khi gán dữ liệu, rồi xác thực tên, điểm. Nhập sai cần báo lỗi thay vì ghi nhầm.

Thuật ngữ cần nhớ: Create (thêm mới).

Câu 3

Duyệt `.items()` để lấy cặp ID–thông tin và in nội dung. Giữ nguyên thứ tự thêm vào của từ điển.

Vì sao duyệt `items()`? Dictionary ngoài chứa ID còn dictionary trong chứa tên, điểm. Cần lấy cả khóa và dữ liệu cho mỗi bản ghi. **Các bước:** Kiểm tra có sinh viên không → duyệt từng cặp ID/bản ghi → truy cập tên, điểm → in từng dòng.

```
for student_id, info in students.items():
    print(student_id, info["name"], info["score"])
```

Giải thích: `.items()` trả các cặp (khóa, giá trị), `for` lặp lần lượt, `info["score"]` lấy điểm. Không ép dictionary thành list để tìm kiếm nếu chỉ cần hiển thị.

Điểm cần kiểm tra: thao tác CRUD thêm, đọc, sửa và xóa sau khi xác nhận.

Ví dụ, cách kiểm tra và lỗi dễ gặp: items() cho phép lấy cả mã và thông tin; in đủ ID, tên và điểm. Danh sách rỗng cần trả thông báo dễ hiểu.

Thuật ngữ cần nhớ: Read (đọc dữ liệu).

Câu 4

Từ ID đầu vào, tìm đúng khóa; từ chối ID không tồn tại, xác thực giá trị mới trước khi thay thế.

Vì sao phải kiểm tra ID trước khi sửa? Không được vô tình tạo một sinh viên mới khi người dùng định cập nhật một sinh viên đang có. **Các bước:** Nhận ID → tìm bản ghi → xác thực tên/điểm mới → thay trường tương ứng → giữ nguyên khóa ID.

```
student_id = "S01"
new_score = 9.0
if student_id not in students:
    raise KeyError("Không tìm thấy sinh viên")
if 0 <= new_score <= 10:
    students[student_id]["score"] = new_score
```

Cú pháp: `not in` kiểm tra vắng mặt; `KeyError` báo khóa không có; `0 <= ... <= 10` giới hạn điểm; phép gán vào `["score"]` chỉ cập nhật trường điểm, không thay dictionary khác.

Điểm cần kiểm tra: sắp xếp điểm và tổng hợp bằng `Counter`.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Chỉ thay đổi bản ghi có đúng ID; nếu không tìm thấy mã phải xử lý chứ không tự tạo người mới.

Thuật ngữ cần nhớ: Update (cập nhật).

Câu 5

Tìm ID rồi kiểm tra `confirmed`; khi người dùng từ chối thì không đụng đến dữ liệu. Khi đồng ý mới `del students[id]`.

Vì sao cần xác nhận xóa? Xóa một khóa sẽ mất toàn bộ dữ liệu của người đó, nên chỉ xóa khi đúng ID và đã xác nhận. **Các bước:** Kiểm tra ID tồn tại → đọc xác nhận → chỉ gọi xóa khi được đồng ý → kiểm tra kết quả.

```
student_id = "S01"
confirmed = True
if student_id in students and confirmed:
    del students[student_id]
```

Cú pháp: `and` chỉ đúng khi **cả hai** điều kiện đúng, `del students[id]` xóa khóa; `confirmed` là Boolean, tương ứng tham số `confirmed=False` mặc định trong hàm hoàn chỉnh để không xóa ngoài ý muốn.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Chỉ xóa khi tìm thấy mã và người dùng đã xác nhận; thử hủy xóa để chứng minh dữ liệu giữ nguyên.

Thuật ngữ cần nhớ: Delete (xóa).

Câu 6

Phân biệt kiểm tra kiểu dữ liệu và kiểm tra khoảng giá trị; nêu rõ vì sao `input()` tạo chuỗi, vì sao cần bắt `ValueError`.

Vì sao kiểm tra dữ liệu ở một chỗ? Các chức năng Add và Update cần dùng cùng quy tắc; viết rải rác dễ chỗ cho điểm âm, chỗ lại không. **Các bước:** Kiểm tra ID/tên không rỗng → chuyển điểm sang số → kiểm tra miền 0–10 → báo lỗi rõ ràng nếu sai.

```
def valid_score(score):
    try:
        number = float(score)
    except ValueError:
        return False
    return 0 <= number <= 10
```

Cú pháp: `try/except ValueError` bắt lỗi chuyển chuỗi không phải số; `return` trả kết quả Boolean; <= gồm cả 0 và 10. Trong code cuối bài, hàm `validate` còn kiểm tra ID và tên.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: strip() bỏ khoảng trắng ở tên; float() chuyển điểm sang số; kiểm tra miền điểm của code nguồn.

Thuật ngữ cần nhớ: Validate (xác thực dữ liệu).

Câu 7

Dùng cùng hàm `add` trong code nguồn để chống trùng trước thao tác gán: nếu `id in students` thì báo lỗi.

Vì sao ID phải duy nhất? Hai sinh viên có thể trùng tên nhưng vẫn cần hai hồ sơ tách biệt; dùng tên làm khóa có nguy cơ ghi đè. **Các bước:** Chuẩn hóa mã → kiểm tra `in` để xác định đã có → chỉ lưu ID chưa xuất hiện → xử lý trường hợp nhập trùng.

```
student_id = "S02"
already_exists = student_id in students
if already_exists:
    print("Mã đã được sử dụng")
```

Giải thích: `in` trên dictionary kiểm tra **khóa**, không kiểm tra các tên nằm bên trong; `already_exists` lưu giá trị đúng/sai; khóa duy nhất là quy tắc của cấu trúc dữ liệu, không phải hàm tính toán.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Không kiểm tra trùng bằng tên, bởi hai sinh viên có thể có tên giống nhau; cần kiểm tra khóa ID.

Thuật ngữ cần nhớ: duplicate ID (trùng mã).

Câu 8

Sắp theo khóa `score`, dùng `reverse=True/False` để đổi chiều; chương trình không làm biến đổi dữ liệu đang lưu.

Vì sao truyền `key` cho `sorted`? Cần xếp theo **điểm**, không phải theo ID hoặc tên. **Các bước:** Lấy các cặp ID và thông tin → lấy trường điểm để so sánh → chọn tăng/giảm → trả danh sách có thứ tự mới.

```
ascending = sorted(students.items(), key=lambda pair: pair[1]["score"])
descending = sorted(students.items(), key=lambda pair: pair[1]["score"], reverse=True)
```

Giải thích flag: `key=lambda pair: pair[1]["score"]` lấy điểm từ dictionary ở vị trí thứ hai của cặp; `reverse=True` đảo sang giảm dần, mặc định `False` tăng dần. `sorted` không thay đổi dictionary gốc, kết quả là list các cặp.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: sorted() sắp điểm số tăng/giảm; điểm là float chứ không phải chuỗi chữ, tránh thứ tự sai.

Thuật ngữ cần nhớ: Sorting (sắp xếp).

Câu 9

Lấy danh sách điểm; tính `max`, `min`, trung bình, rồi đếm nhãn phân loại bằng `Counter`. Cần xử lý trường hợp không có sinh viên để tránh chia 0.

Vì sao tính thống kê trước khi phân loại? Trung bình tóm tắt kết quả toàn nhóm, còn đếm theo loại cho thấy có bao nhiêu người ở mỗi mức; không thể lấy số lượng chia cho 0 nếu chưa có sinh viên. **Các bước:** Lấy danh sách điểm → tính tổng và số lượng → tìm trung bình → gọi hàm `classify` cho từng người → đếm nhãn.

```
from collections import Counter
scores = [row["score"] for row in students.values()]
average = sum(scores) / len(scores) if scores else None
levels = Counter(classify(score) for score in scores)
```

Giải thích: `.values()` lấy bản ghi không cần ID; `sum` cộng điểm, `len` đếm; biểu thức `... if scores else None` tránh chia 0; `Counter` đếm số lần xuất hiện của từng nhãn. `classify` là hàm định nghĩa trong code hoàn chỉnh, không phải hàm Python có sẵn.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Trung bình bằng tổng điểm/số người; cần xử lý n=0 để tránh chia cho 0; Counter đếm mức học lực.

Thuật ngữ cần nhớ: Statistics (thống kê).

Câu 10

Kiểm tra ngưỡng từ cao xuống thấp để không có hai mức cùng đúng; các mốc lấy từ code tham khảo đã tồn tại, **không giả nhận là đề Word quy định**.

Vì sao xét mức điểm từ cao xuống thấp? Nếu xét ngưỡng thấp trước, sinh viên 9.5 điểm sẽ khớp cả mức 5 và 7, dễ bị phân loại sai. Các nhánh `if/elif` sẽ dừng ở nhánh đầu thỏa.
Các bước: Kiểm tra điểm hợp lệ → so ngưỡng 9 → 7 → 5 → còn lại mức Poor.

```
if score >= 9:
    level = "Excellent"
elif score >= 7:
    level = "Good"
elif score >= 5:
    level = "Average"
else:
    level = "Poor"
```

Cú pháp: >= nghĩa là lớn hơn hoặc bằng; `elif` chỉ kiểm tra khi các điều kiện trước sai; `else` là phần còn lại. Vì thứ tự kiểm tra từ cao xuống, không cần viết lặp cả cận dưới/cận trên. Các ngưỡng thuộc quy tắc của code tham khảo.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Các ngưỡng học lực là quy ước của code tham khảo, không mặc định là quy định từ đề Word.

Thuật ngữ cần nhớ: classification (phân loại).